



Algoritmer för schemaläggning med många delade resurser

Implementering och analys av en exakt algoritm

Algorithms for scheduling with many shared resources

Examensarbete för kandidatexamen i matematik vid Göteborgs universitet

Ida Gustafsson

Rebecca Runesson

Alma Sjölin

Algoritmer för schemaläggning med många delade resurser

Implementering och analys av en exakt algoritm

Examensarbete för kandidatexamen i matematik inom Matematikprogrammet, inriktning Tillämpad matematik, vid Göteborgs universitet

Ida Gustafsson Rebecca Runesson Alma Sjölin

Handledare: Malin Rau
 Albert Vesterlund

Institutionen för Matematiska vetenskaper
CHALMERS TEKNISKA HÖGSKOLA
GÖTEBORGS UNIVERSITET
Göteborg, Sverige 2026

Förord

Bidragsrapport

Ida Gustafsson - Har haft huvudansvaret för att utveckla algoritmerna för maskintilldelningen och testgeneratoren. I detta arbete har det även ingått att skriva pseudokod och bevis för algoritmer-nas korrekthet (Avsnitt 3.1, Avsnitt 5.1.1 och Appendix A.1). Läst på om och skrivit delarna om simulated annealing, relaterat arbete och symmetrier (Avsnitt 5.1.3, Avsnitt 1.2 och Avsnitt 6.1). Utfört alla tester för testserierna c , f och h , samt visualiserat resultaten och skrivit diskussionsdel tillhörande testserie h (Figur 11 och Avsnitt 5.3.3). Skrivit övriga delar i Avsnitt 5.1 och Avsnitt 3, med undantag för Avsnitt 3.2, samt gjort tabellerna i Appendix A.2. Implementerat maskintilldelningen, testgeneratoren och LB-optimeringen i Python samt skrivit en förklaring av koden i form av en README-fil på GitHub.

Rebecca Runesson - Har haft huvudansvaret för att skriva förklaring och korrekthetsbevis för ILP-formuleringen (Avsnitt 3.2). Skrivit diskussionsdelen (Avsnitt 5.3), med undantag för Avsnitt 5.3.3, samt läst på om och skrivit delarna om IC- och CMB-optimeringarna (Avsnitt 4.3 och Avsnitt 4.4). Utformat Tabell 1 och skrivit sammandrag och abstract. Utfört alla tester för testserierna b och e samt implementerat IC-optimeringen i Python.

Alma Sjölin - Har skrivit inledning (med undantag för Avsnitt 1.2), vilket innefattat den matematiska formuleringen av MSRS-problemet (Avsnitt 1.1). Läst på om och skrivit delarna om LB- och WS-optimeringarna samt samhällliga aspekter (Avsnitt 4.1, Avsnitt 4.2 och Avsnitt 2). Utfört alla tester för testserierna a , d och g och skapat graferna i Figur 2 till 9 samt stapeldiagrammet i Figur 10. Implementerat ILP:n samt WS- och CMB-optimeringarna i Python och bidragit till ett första utkast av testgeneratoren. Studerat och skrivit om förbättringar av WS-optimeringen (Avsnitt 6.2) samt implementerat dessa i Python.

Ovan framgår vem som huvudsakligen gjort vad i projektet. Utöver detta har alla hjälpts åt att finslipa varandras delar och gemensamt löst problem som dykt upp. Under projektets gång har det även förts en gemensam loggbok över nedlagd tid och utförda uppgifter.

Sammandrag

Denna rapport behandlar problemet om schemaläggning med många delade resurser, där en mängd jobb ska schemaläggas på identiska parallella maskiner med målet att minimera tiden då sista jobbet avslutas. Varje jobb kräver tillgång till en delad resurs och jobb som är beroende av samma resurs kan inte bearbetas samtidigt. Vi studerar en exakt lösningsmetod baserad på en linjär heltalsmodell (ILP) som bestämmer den optimala starttiden för varje jobb. Utifrån dessa starttider konstrueras en maskintilldelning i polynomiell tid. För att förbättra körtiden undersöks flera optimeringsstrategier: undre begränsningar, warm start baserad på en $\frac{3}{2}$ -approximationsalgoritm, indicator constraints samt en kombination av dessa metoder. Metoderna utvärderas genom beräkningsexperiment på genererade instanser med varierande parametrar, där den optimala lösningen är känd för varje instans. Resultaten visar en jämförelse av den exakta lösningsmetoden utan optimeringar med de olika optimeringsmetoderna och att warm start ger den bästa förbättringen av körtid och lösningskvalitet. Vidare jämför vi warm start-metoden med den heuristiska metoden simulated annealing, vilket indikerar att heuristiska metoder är mer lämpliga för storskaliga instanser. Slutligen analyserar vi hur instansegenskaper, mer precist bearbetningstider, påverkar körtiden.

Abstract

This report considers the many shared resources scheduling problem, where a set of jobs needs to be scheduled on identical parallel machines with the objective of minimizing the completion time of the last job. Each job requires access to one shared resource, and jobs that depend on the same resource cannot be processed simultaneously. We study an exact solution method based on an integer linear programming (ILP) model that determines the optimal start time for each job. From these start times a machine assignment is constructed in polynomial time. To improve running time, several optimization strategies are examined: lower bounds, warm start derived from a $\frac{3}{2}$ -approximation algorithm, indicator constraints and a combination of these approaches. The methods are evaluated through computational experiments on generated instances, with varying parameters, where the optimal solution is known for each instance. The results show a comparison of the exact solution method without optimizations and the various optimization methods, demonstrating that warm start provides the best improvement in running time and solution quality. Furthermore, we compare the warm start method with the heuristic method simulated annealing, which indicates that heuristic methods are more suitable for large scale instances. Finally, we analyse how instance characteristics, more precisely processing times, affect running time.

Innehåll

1	Inledning	1
1.1	MSRS-problemet	1
1.2	Relaterat arbete	2
2	Samhälleliga aspekter	2
3	Program för exakt lösning av MSRS-problemet	3
3.1	Maskintilldelning σ	3
3.2	Starttider t	3
3.3	Generering av det fullständiga schemat (σ, t)	5
4	Optimering av programmet	5
4.1	Lägre begränsning	5
4.2	Warm Start	5
4.3	Indicator constraints	6
4.4	Kombination av samtliga optimeringar	6
5	Prestandaanalys	6
5.1	Metod	6
5.1.1	Testgenerator	6
5.1.2	Schema för generering av testfiler	7
5.1.3	Simulated annealing	8
5.2	Resultat	9
5.2.1	Effekt av optimeringar	9
5.2.2	Prestanda för warm start och simulated annealing vid stora instanser . . .	14
5.3	Diskussion	14
5.3.1	Den bästa optimeringsmetoden	14
5.3.2	Variation i bearbetningstid och dess påverkan på körtiden	16
5.3.3	Avvägning mellan exakt och approximativ lösningsmetod	16
6	Vidare forskning	16
6.1	Symmetrier i ILP-formuleringen	16
6.2	Vidare optimering av Warm Start	17
	AI-användning	19
A	Appendix	i
A.1	Pseudokod för testgeneratorn	i
A.2	Testresultat	ii

1 Inledning

Schemaläggning är ett klassiskt kombinatoriskt optimeringsproblem med tillämpningar inom en mängd olika områden, från industriell produktion till rymdfart. I detta arbete studerar vi en specifik variant kallad *many shared resources scheduling* (MSRS), ett problem där ett antal jobb med olika bearbetningstider ska schemaläggas på ett antal identiska maskiner, under restriktionen att jobb som delar samma resurs, och därmed tillhör samma klass, inte får bearbetas parallellt.

En välkänd tillämpning av MSRS-problemet är nedladdning av observationsdata från jordobservationssatelliter. Data som samlas in lagras ombord i ett begränsat antal minnesbanker, vilka utgör de delade resurserna, medan nedladdningskanalerna till en markstation utgör maskinerna. Varje kanal hanterar nedladdningar sekventiellt och varje fil kan laddas ned via vilken kanal som helst. Under det begränsade tidsfönster då satelliten har kontakt med markstationen är det av intresse att ladda ned all lagrad data så snabbt som möjligt, utan att bryta mot resursrestriktionerna. Detta formuleras som ett optimeringsproblem med målet att minimera det som kallas *makespan*, den tidpunkt då det sista jobbet avslutas [10].

MSRS-problemet är NP-svårt, vilket innebär att det tillhör en klass av problem för vilka ingen känd algoritm existerar som löser dem exakt inom polynomiell tid, under antagandet att $P \neq NP$. Polynomiell tid innebär att körtiden växer polynomiskt i förhållande till instansstorleken. Eftersom problemet är NP-svårt har approximationsalgoritmer tidigare studerats och tillämpats inom detta område. För en instans I av ett minimeringsproblem betecknar vi det optimala målfunktionsvärdet med $\text{OPT}(I)$. En algoritm A är en α -approximation om den för varje instans I garanterar en lösning vars målfunktionsvärde $A(I)$, uppfyller $A(I) \leq \alpha \cdot \text{OPT}(I)$, där faktorn $\alpha \geq 1$ kallas *approximationskvot*.

Detta projekt fokuserar på att lösa MSRS-problemet exakt och fastställa för vilka instansstorlekar detta är praktiskt möjligt. Syftet är att undersöka avvägningen mellan lösningstid och lösningskvalitet. Detta uppnås genom att implementera ett program som löser problemet exakt med hjälp av optimeringsverktyget Gurobi. Därefter testas och utvärderas olika strategier för att optimera detta program med målet att reducera dess körtid. Slutligen fastställs vilken av optimeringarna som är mest effektiv. Denna version av programmet jämförs sedan med den heuristiska metoden *simulated annealing* [13], för att utvärdera när en exakt lösningsmetod är att föredra framför en approximativ.

1.1 MSRS-problemet

MSRS-problemet definieras på följande vis. Låt m vara antal maskiner (nedladdningskanaler), $\mathcal{J}$ en mängd jobb (data som ska laddas ner, med $|\mathcal{J}| = n$) samt $\mathcal{R}$ en mängd resurser (minnesbanker), där $m \leq |\mathcal{R}| \leq n$. Resurserna bildar en partition $\mathcal{C}$ av $\mathcal{J}$ med $|\mathcal{C}| = K$ stycken klasser $c \in \mathcal{C}$, där varje klass motsvarar mängden jobb som delar samma resurs $r \in \mathcal{R}$. Jobb som tillhör samma klass kan inte bearbetas parallellt, då varje jobb kräver exklusiv tillgång till sin resurs under bearbetningen. Varje jobb $j \in \mathcal{J}$ har en bearbetningstid $p_j \in \mathbb{R}$ och ska tilldelas en starttid t_j . Ett schema $(\sigma, t) : \mathcal{J} \rightarrow [m] \times \mathbb{R}_{\geq 0}$ är en tilldelning som för varje jobb $j \in \mathcal{J}$ tilldelar en maskin $\sigma(j) \in [m]$ och en starttid $t(j) \in \mathbb{R}_{\geq 0}$. Starttiden $t(j)$ för $j \in \mathcal{J}$ betecknas fortsättningsvis med t_j . Makespan för ett schema är därmed $\max_{j \in \mathcal{J}}(t_j + p_j)$. Ett schema sägs vara giltigt om följande två restriktioner är uppfyllda:

- (*) **Maskinrestriktion:** Två jobb som körs på samma maskin överlappar aldrig. Det vill säga, för två jobb i, j ($i \neq j$), där $\sigma(j) = \sigma(i)$ så gäller $[t_j, t_j + p_j) \cap [t_i, t_i + p_i) = \emptyset$.
- (\diamond) **Resursrestriktion:** Två jobb från samma klass körs aldrig parallellt. Det vill säga, för två jobb $i, j \in c$ ($i \neq j$) för någon klass $c \in \mathcal{C}$ gäller att $[t_i, t_i + p_i) \cap [t_j, t_j + p_j) = \emptyset$.

Målet är att bestämma det schema som minimerar makespan, det vill säga $\min_{(\sigma, t)} \max_{j \in \mathcal{J}}(t_j + p_j)$.

1.2 Relaterat arbete

Schemalägningsproblem med delade resurser har studerats under lång tid. 1983 introducerades en klassificering av schemalägningsproblem med resursbivillkor [1]. I samband med detta etablerades den så kallade *3-field notation*, i vilken MSRS-problemet betecknas som $P|res \cdot 111|C_{\max}$. Själva MSRS-problemet formulerades däremot först 2016 [10], motiverat av den praktiska utmaningen att planera nedladdning av satellitdata. Då presenterades en $(2 - \frac{2}{m+1})$ -approximation för instanser med m maskiner. Denna approximation konvergerar mot 2 då antalet maskiner ökar, vilket länge utgjorde det bästa kända resultatet för det allmänna fallet.

Under senare år har dessa resultat förbättrats. 2018 redogjordes det för en *polynomial time approximation scheme* (PTAS) för fallet med ett konstant antal maskiner, vilken även är tillämplig på det mer generaliserade problemet där jobb är begränsade till specifika delmängder av maskinerna [4]. Vidare utvecklades 2021 en snabbare och enklare version av den ursprungliga $(2 - \frac{2}{m+1})$ -approximationen, samt en $\frac{6}{5}$ -approximation för specialfallet med två maskiner [16]. De hittills bästa resultaten för MSRS-problemet publicerades 2023 [3]. Detta inkluderade en $\frac{5}{3}$ - och $\frac{3}{2}$ -approximation, samt en PTAS för ett konstant antal maskiner som förbättrade det tidigare resultatet i [4].

2025 formulerades en ny $\frac{4}{3}$ -approximation för instanser utan stora jobb [12]. Utöver detta implementerades och optimerades $\frac{5}{3}$ - och $\frac{3}{2}$ -approximationerna från [3], varpå dessa jämfördes mot heuristiska lösningsmetoder.

2 Samhälleliga aspekter

Jordobservationssatelliter samlar kontinuerligt in stora mängder data om klimat, vegetation, hav och naturkatastrofer, som sedan måste laddas ned till markstationer inom snäva tidsfönster. Effektiv schemaläggning av denna nedladdning har därmed direkta konsekvenser för beslut inom exempelvis katastrofberedskap, miljöövervakning och väderprognoser.

Copernicus, EU:s jordobservationsprogram, är ett konkret exempel på hur satellitdata skapar samhällsnytta i stor skala. Programmet erbjuder tjänster inom atmosfär, hav, land, klimat, katastrofin-satser och säkerhet och levererar dagligen flera terabyte öppna data till hundratusentals användare [6]. Datan används bland annat av IPCC som stöd för fysiska bevis kring havsnivåer, växthusgaser och havsis, och spelar även en viktig roll vid akuta samhällskriser [15]. För att dessa tjänster ska fungera effektivt krävs det att de stora datamängderna som satelliterna samlar in kan laddas ned till markstationer på ett tillförlitligt och snabbt sätt. Det är just här som schemalägningsproblem av den typ som studeras i detta projekt spelar en direkt roll. Effektivare schemalägningsalgoritmer för datanedladdning kan därmed direkt påverka hur snabbt satellitdata når beslutsfattare och räddningstjänst, vilket i en krissituation kan vara avgörande för hur snabbt och effektivt åtgärder kan sättas in.

Satelliters förmåga att samla in data är inte begränsad till civila ändamål. Dessa utgör exempel på teknik med så kallade dubbla användningsområden, det vill säga teknik som kan nyttjas för både civila och militära ändamål, vilket är särskilt relevant inom rymdsektorn. Copernicus driver exempelvis en dedikerad säkerhetstjänst för gräns- och havsövervakning vid EU:s yttre gränser [2]. Optimerad schemaläggning kan därmed indirekt effektivisera sådana system, oavsett om de används för klimatövervakning eller militär underrättelseinhämtning.

Effektivare schemaläggning kan också bidra till bättre resursutnyttjande och därmed potentiellt förlänga satelliters livslängd. Detta minskar behovet av att bygga och skjuta upp nya satelliter, vilket är positivt ur både ett ekonomiskt och miljömässigt perspektiv. Färre satelliter i omloppsbana bidrar dessutom till att hålla rymden tillgänglig för framtida uppdrag, vilket är centralt för en långsiktigt hållbar rymdsektor [5].

3 Program för exakt lösning av MSRS-problemet

Detta avsnitt beskriver ett program som löser MSRS-problemet exakt. Programmet skapar ett schema (σ, t) genom att först generera starttiderna t och sedan maskintilldelningen σ . Enligt Sats 3.1 kan σ genereras i polynomiell tid givet giltigt t . Det är alltså genereringen av starttiderna som är den beräkningsmässigt mest komplexa delen av programmet. Vi kommer i det följande bevisa att schemat (σ, t) är optimalt och uppfyller restriktionerna $(\star)$ och $(\diamond)$.

3.1 Maskintilldelning σ

Låt t vara ett godtyckligt starttidsschema som uppfyller $(\diamond)$ och $|\{j \in \mathcal{J} \mid t_j \leq \tau < t_j + p_j\}| \leq m$ för varje tidpunkt $\tau \in \mathbb{R}$. Det innebär att jobb från samma klass inte körs parallellt och att högst m jobb körs samtidigt. Givet t fastställer *Algorithm 1* maskintilldelningen σ genom att iterativt schemalägga jobb utefter starttid. Varje jobb tilldelas den maskin som för tillfället är tidigast tillgänglig. Formellt definierar vi $C(k)$ som sluttiden för det senast schemalagda jobbet på maskin k i det aktuella schemat.

Algorithm 1: Maskintilldelning

Indata: t **Utdata:** (σ, t) Sortera t så att $t_1 \leq t_2 \leq \dots \leq t_n$ **for** $t_j \in t$ **do** $i \leftarrow \arg \min_{k \in [m]} C(k)$ Schemalägg jobb j på maskin i med starttid t_j **return** (σ, t)

Sats 3.1. *Algorithm 1 genererar en giltig maskintilldelning σ i polynomiell tid.*

Bevis. Eftersom *Algorithm 1* endast itererar över jobben en gång är tidskomplexiteten polynomiell. Vidare gäller $\min_{k \in [m]} C(k) = 0$ innan totalt m jobb schemalagts. Därmed schemaläggs de första m jobben på tomma maskiner, vilket utgör en giltig maskintilldelning för alla jobb $j \leq m$.

Antag för motsägelse att σ inte är en giltig maskintilldelning och låt j vara det första ogiltigt schemalagda jobbet i ordningen. Då startar jobb j innan dess tilldelade maskin i blir ledig, dvs. $t_j < C(i)$, där $i = \arg \min_{k \in [m]} C(k)$ efter schemaläggning av jobb $j - 1$.

Eftersom maskintilldelningen är giltig för de m första jobben, måste $j > m$. Att j är det första ogiltigt schemalagda jobbet innebär också per definition att alla tidigare jobb $1, \dots, j - 1$ har schemalagts korrekt och inte överlappar varandra. Då alla tidigare jobb startar senast vid tidpunkten t_j ($t_l \leq t_j$ för alla $l < j$), $j > m$ och i är den maskin som blir ledig först, innebär $t_j < C(i)$ att samtliga m maskiner är upptagna vid tidpunkten t_j . Om jobb j schemaläggs vid t_j kommer därför minst $m + 1$ jobb att köras parallellt. Detta motsäger antagandet om att högst m jobb körs parallellt vid varje tidpunkt, och därmed måste $t_j \geq C(i)$ gälla. □

3.2 Starttider t

Vi formulerar problemet att hitta ett starttidsschema t för alla jobb som ett linjärt heltalsproblem (*integer linear programming*, ILP). Modellen representerar starttider för alla jobb, maskinkapacitet och restriktioner för delade resurser, med målet att minimera makespan.

Vi introducerar följande variabler i modellen. Låt $T \in \mathbb{R}_+$ beteckna makespan. För varje jobb $j \in \mathcal{J}$ definieras starttiden $t_j \in \mathbb{R}_+$. Vidare införs binära variabler $x_{j,j'}, y_{j,j'}, z_{j,j'} \in \{0, 1\}$ där $x_{j,j'} = 1$ om $t_{j'} \leq t_j$, $y_{j,j'} = 1$ om $t_j < t_{j'} + p_{j'}$ och $z_{j,j'} = 1$ om $t_j \in [t_{j'}, t_{j'} + p_{j'})$.

Modellen använder följande parametrar. För varje jobb $j \in \mathcal{J}$ betecknar $p_j \in \mathbb{R}$ bearbetningstiden. Konstanten M motsvarar en övre begränsning på makespan T och väljs stor nog för att säkerställa

att vissa restriktioner är inaktiva när den motsvarande binära variabeln är noll. Konstanten $\varepsilon > 0$ väljs tillräckligt liten för att hantera strikt olikhet i modellen. Därmed kan vi formulera följande linjära heltalsprogram:

$$\begin{aligned}
\min \quad & T \\
& T \leq M & (1) \\
& t_j + p_j \leq T, & \forall j \in \mathcal{J} & (2) \\
& t_{j'} \leq t_j + (1 - x_{j,j'})M, & \forall j, j' \in \mathcal{J} & (3) \\
& t_j + \varepsilon \leq t_{j'} + x_{j,j'}M, & \forall j, j' \in \mathcal{J} & (4) \\
& t_j + \varepsilon \leq t_{j'} + p_{j'} + (1 - y_{j,j'})M, & \forall j, j' \in \mathcal{J} & (5) \\
& t_{j'} + p_{j'} \leq t_j + y_{j,j'}M, & \forall j, j' \in \mathcal{J} & (6) \\
& z_{j,j'} \leq y_{j,j'}, & \forall j, j' \in \mathcal{J} & (7) \\
& z_{j,j'} \leq x_{j,j'}, & \forall j, j' \in \mathcal{J} & (8) \\
& x_{j,j'} + y_{j,j'} - 1 \leq z_{j,j'}, & \forall j, j' \in \mathcal{J} & (9) \\
& \sum_{j' \in \mathcal{J}} z_{j,j'} \leq m, & \forall j \in \mathcal{J} & (10) \\
& z_{j,j'} = 0, & \forall j, j' \in \mathcal{C}, j \neq j', \mathcal{C} \in \mathcal{C} & (11) \\
& t_j \geq 0, & \forall j \in \mathcal{J} \\
& x_{j,j'}, y_{j,j'}, z_{j,j'} \in \{0, 1\}, & \forall j, j' \in \mathcal{J}
\end{aligned}$$

Villkor (1) sätter en övre begränsning för T och (2) säkerställer att T är minst lika stort som jobbet med störst sluttid. Eftersom målet är att minimera T kommer alltså T att sättas exakt som jobbet med störst sluttid. De binära variablerna $x_{j,j'}$, $y_{j,j'}$ och $z_{j,j'}$ täcker alla möjliga tidsmässiga relationer mellan jobben.

Variabel $x_{j,j'}$ anger om jobb j' startar innan eller samtidigt som j (startordning), vilket modelleras genom villkoren (3) och (4). Om $x_{j,j'} = 1$ ger (3) $t_{j'} \leq t_j$, det vill säga att jobb j' startar innan eller samtidigt som jobb j . Om $x_{j,j'} = 0$ ger (4) istället $t_j + \varepsilon \leq t_{j'}$, vilket innebär att j' startar strikt efter j . Variabeln $y_{j,j'}$ beskriver om j startar innan j' har avslutats (överlappning) enligt (5) och (6). Om $y_{j,j'} = 1$ startar j strikt innan j' är klart, medan om $y_{j,j'} = 0$ startar j samtidigt som eller efter att j' är klart. Variabeln $z_{j,j'}$ används för att kontrollera antalet parallella jobb och resurskonflikter. Villkor (7) och (8) säkerställer att $z_{j,j'} = 1$ endast om både $x_{j,j'} = 1$ och $y_{j,j'} = 1$. Villkor (9) ser till att om både $x_{j,j'} = 1$ och $y_{j,j'} = 1$ så måste $z_{j,j'} = 1$. Med andra ord är $z_{j,j'} = 1$ om och endast om j och j' överlappar. Slutligen begränsar villkor (10) antalet parallella jobb till antalet maskiner m , medan villkor (11) begränsar antalet parallella jobb i samma klass, vilket förhindrar att resurser delas samtidigt.

Det återstår därmed att visa att ILP-modellen korrekt representerar MSRS-problemet. Det vill säga, att det finns en ekvivalens mellan en optimal lösning för modellen och ett optimalt starttids-schema, och att makespan bevaras.

Sats 3.2. *En optimal lösning till ILP-modellen motsvarar ett optimalt starttidsschema för MSRS-problemet med samma makespan.*

Bevis. Vi kommer att bevisa detta i två steg: i) ett giltigt schema ger en genomförbar ILP-lösning med samma makespan, och sedan ii) varje genomförbar ILP-lösning motsvarar ett giltigt schema med samma makespan.

i) Antag att vi har ett giltigt schema. Sätt t_j starttid och $T = \max(t_j + p_j)$. Definiera de binära variablerna enligt deras betydelse: $x_{j,j'} = 1$ om jobb j' startar innan eller samtidigt som jobb j , $y_{j,j'} = 1$ om jobb j startar innan jobb j' är klart och $z_{j,j'} = 1$ om jobben överlappar. Eftersom schemat är giltigt körs högst m jobb samtidigt och inga jobb i samma klass körs samtidigt. Alltså uppfylls alla restriktioner och därmed är ILP-modellen genomförbar och har samma makespan T .

ii) Antag att vi har en genomförbar ILP-lösning. Definiera schemat genom att starta varje jobb vid t_j . Då gäller att (2) säkerställer korrekt sluttid. (10) säkerställer att högst m jobb körs parallellt och (11) ser till att inga jobb i samma klass överlappar. Alltså är schemat giltigt och har makespan T . Optimalitet följer från i) och ii) då mängden av alla giltiga starttidsscheman är ekvivalent med mängden av alla genomförbara ILP-lösningar där makespan är identiska, vilket ger att en optimal lösning till ILP-modellen tillsammans med en giltig maskintilldelning givet av *Sats 3.1*, motsvarar en optimal lösning för MSRS-problemet. $\square$

3.3 Generering av det fullständiga schemat (σ, t)

Vårt program för exakt lösning av MSRS-problemet genererar schemat (σ, t) i två steg. Först fastställs starttiderna t genom att lösa ILP:n i Avsnitt 3.2 med optimeringsverktyget Gurobi. Därefter tillämpas *Algoritm 1* för att generera en giltig maskintilldelning σ givet t . Enligt *Sats 3.1* och *Sats 3.2* är (σ, t) optimalt och uppfyller restriktionerna $(\star)$ och $(\diamond)$.

4 Optimering av programmet

Eftersom maskintilldelningen σ kan bestämmas i polynomiell tid, ligger fokus för potentiella optimeringar på starttidsschemat t .

4.1 Lågre begränsning

Att förse Gurobi med en undre begränsning L för makespan T kan påskynda lösningsprocessen. Utan en undre begränsning behöver Gurobi utforska hela intervallet $[0, M]$, medan begränsningen $T \geq L$ direkt utesluter alla lösningar med makespan mindre än L . Detta krymper sökrymden till intervallet $[L, M]$ och gör att Gurobi snabbare kan fokusera sin sökning mot optimala lösningar. Den undre begränsningen L som används bygger på två observationer: att jobb i samma klass måste schemaläggas sekventiellt, samt att minst två av de $m + 1$ största jobben måste schemaläggas på samma maskin. Den undre begränsningen formuleras enligt *Sats 4.1*:

Sats 4.1. Låt $p_{(1)} \geq p_{(2)} \geq \dots \geq p_{(n)}$ vara bearbetningstiderna för alla jobb $j \in \{1, \dots, n\}$ sorterade i avtagande ordning. Då är

$$L = \max \left(\max_{c \in C} \sum_{j \in c} p_j, p_{(m)} + p_{(m+1)} \right)$$

en undre begränsning för makespan i varje giltigt schema för MSRS-problemet.

Bevis. Makespan måste vara minst lika stort som den största klassens totala bearbetningstid, eftersom jobb i samma klass inte får schemaläggas parallellt. Vidare är $p_{(m)} + p_{(m+1)}$ en undre begränsning eftersom två av de $m + 1$ största jobben måste schemaläggas på samma maskin, och $p_{(m)} + p_{(m+1)}$ utgör en undre begränsning för bearbetningstiden för dessa två jobb. $\square$

4.2 Warm Start

Ytterligare en potentiell optimering är att förse Gurobi med startvärden från en approximationsalgoritm. Denna typ av optimering kallas för *warm start* och innebär att lösaren ges en initial lösning som utgångspunkt, vilket kan hjälpa den att snabbare hitta en optimal lösning. För heltalsprogrammeringsproblem används mer specifikt termen *MIP start*, och en sådan kan förbättra lösningstiden på flera sätt. Den initiala lösningen används som en övre gräns på målfunktionen, vilket gör att lösaren kan utesluta delar av sökträdet där lösningarna inte kan bli bättre än den redan kända lösningen och därmed fokusera sökningen mot mer lovande områden.

Problemet löses först med $\frac{3}{2}$ -with-enqueue-approximationsalgoritmen beskriven i [12], vilken valdes då den presterade bäst i de flesta av de genomförda testerna bland de implementerade approximationsalgoritmerna. Starttiderna t_j för varje jobb $j \in \mathcal{J}$ samt makespan används som startvärden i

ILP-modellen. Makespan används dessutom som övre begränsning M , vilket gör att lösaren snabbare kan fokusera sin sökning mot optimala lösningar och utesluta scheman med makespan $T \geq M$. Starttiderna t_j och makespan T ges som en partiell MIP-start till Gurobi, vilket innebär att startvärden endast sätts för en delmängd av modellens variabler. De binära variablerna $x_{jj'}$, $y_{jj'}$ och $z_{jj'}$ lämnas odefinierade, och Gurobi försöker automatiskt härleda lämpliga värden för dessa utifrån de angivna startvärdena [9]. Dessutom används den undre gränsen L , beskriven i Avsnitt 4.1, som en undre begränsning på makespan T i modellen, vilket ytterligare stramar åt sökrymden från båda håll.

4.3 Indicator constraints

Den nuvarande ILP-modellen utgår från big-M-formuleringar av villkoren (3)-(6) där konstanten M används för att aktivera eller avaktivera villkoren beroende på värdet hos de binära variablerna $x_{j,j'}$ och $y_{j,j'}$. Stora värden på big-M kan leda till för svaga relaxationer och i sin tur att problemet tar lång tid att lösa. Med relaxation avses här en förenkling som tillåter de binära variablerna att anta värden i intervallet $[0, 1]$. En möjlig förbättring av modellen i Gurobi är att ersätta dessa big-M-formuleringar med så kallade *indicator constraints*. Då byts exempelvis villkor (3) och (4) ut mot indicator constraint $x_{j,j'} = 1 \Rightarrow t_{j'} \leq t_j$ och $x_{j,j'} = 0 \Rightarrow t_j + \varepsilon \leq t_{j'}$ utan behov av konstanten M [7]. På samma sätt ersätts villkor (5) och (6) med indicator constraints för variabeln $y_{j,j'}$.

4.4 Kombination av samtliga optimeringar

Ytterligare en metod är att kombinera samtliga tidigare beskrivna metoder, alltså lägre begränsning, warm start och indicator constraints. Vi benämner metoden *combined*. Syftet med metoden är att undersöka om kombinationen av olika optimeringsstrategier kan ge en förstärkande effekt: en mindre sökrymd genom warm start samt lägre begränsning och en bättre modellformulering via indicator constraints, och om det i sin tur kan reducera lösningstiden ytterligare. Liksom för warm start löses problemet initialt med $\frac{3}{2}$ -with-enqueue-algoritmen.

5 Prestandaanalys

I detta avsnitt analyseras prestandan hos programmet beskrivet i Avsnitt 3 genom empiriska tester. Det primära syftet är att utvärdera under vilka omständigheter en exakt lösningsmetod är att föredra framför en approximativ, vilket görs genom att jämföra modellen med heuristiken *Simulated Annealing* [13] (beskriven i Avsnitt 5.1.3). Vidare undersöks hur programmets effektivitet påverkas av förhållandet mellan antalet maskiner m , klasser K och jobb n , samt vilken prestandavinst de optimeringar beskrivna i Avsnitt 4 medför.

5.1 Metod

De exakta lösningsmetoderna (standardprogrammet och dess optimerade versioner) är implementerade i Python, och källkod för dessa finns på GitHub¹. I samtliga versioner är ε satt till 0.1 och en övre tidsgräns på 10 minuter tillämpas, varefter den bästa funna lösningen returneras. Simulated annealing-algoritmen är implementerad i C++ (se Avsnitt 5.1.3).

För att säkerställa att prestandajämförelsen blir rättvis krävs instanser med känt optimalt makespan. Testdatan genereras därför med testgeneratoren beskriven i Avsnitt 5.1.1 och samtliga instanser har optimalt makespan $T = 1000$. För att möjliggöra en analys av hur parametrarna m , K och n påverkar prestandan, skapas testfiler enligt schemat i Avsnitt 5.1.2, vilket säkerställer en bred variation på instansegenskaper.

5.1.1 Testgenerator

Som indata tar testgeneratoren antalet jobb n , klasser K , maskiner m samt önskat optimalt makespan T . Utifrån dessa parametrar genereras bearbetningstider $p_j \in \{1, \dots, T\}$ och klasstillhö-

¹<https://github.com/almasjolin/Bachelor-Thesis-MSRS>

righeter $c_j \in \{1, \dots, K\}$ för varje jobb $j = 1, \dots, n$. Att bearbetningstiderna begränsas till heltal innebär ingen förlust av generalitet, då rationella bearbetningstider alltid kan skalas till heltal genom multiplikation med lämplig faktor.

Enligt *Sats 5.1* garanterar generatorns konstruktion att det optimala makespan-värdet för de resulterande instanserna är lika med T . Själva testgenereringen sker i fyra steg, vilka består av att partitionera intervallet $[0, W]$ i exakt n delintervall, där $W = mT$ utgör den totala arbetsmängden (se Figur 1a). Delintervallen motsvarar bearbetningstiderna för jobben och definieras av brytpunkter $b_1, \dots, b_n \in (0, W]$ som markerar var ett jobb slutar och nästa börjar. De fyra stegen beskrivs översiktligt nedan och mer detaljerat i *Algoritm 2* i Appendix A.1.

Steg 1. Inledningsvis definieras K brytpunkter vilka avgränsar varje klass. Dessa genereras slumpmässigt så att bearbetningstiden för varje klass följer en Poisson-fördelning med väntevärde $\mu = \frac{W}{K}$, vilket säkerställer en jämn fördelning av arbetsmängden. För att garantera att en giltig lösning med makespan T existerar begränsas varje klassbearbetningstid till intervallet $[1, T]$. Den sista klassbrytpunkten sätts till W .

Steg 2. Därefter definieras brytpunkter vid varje maskins kapacitetsgräns, dvs. vid $T, 2T$ t.o.m mT .

Steg 3. I steg 1 och 2 definieras d unika brytpunkter där $K \leq d \leq m + K - 1$. Ifall $d \neq n$ utförs följande:

- Om $d > n$ reduceras totala antalet brytpunkter genom att ersätta den i :te minsta klassbrytpunkten med $i \cdot T$ för $i = 1, 2, \dots$, tills exakt n unika brytpunkter kvarstår. Detta säkerställer att bearbetningstiden för varje klass tillhör intervallet $[1, T]$.
- Om $d < n$ utökas totala antalet brytpunkter genom att slumpmässigt definiera ytterligare brytpunkter från mängden $\{1, \dots, W\}$ tills exakt n unika brytpunkter erhållits.

Steg 4. Slutligen sorteras brytpunkterna så att $b_1 < b_2 < \dots < b_n$. För varje jobb $j = 1, \dots, n$ definieras en bearbetningstid $p_j = b_j - b_{j-1}$, och en klass c_j som bestäms av det klassintervall som innefattar b_j .

Sats 5.1. *De testinstanser som genereras av testgeneratorn har optimalt makespan lika med T .*

Bevis. Eftersom $b_j \in (0, W] = (0, mT]$ existerar det för varje jobb $j = 1, \dots, n$ ett unikt maskinindex $k_j \in \{1, \dots, m\}$ sådant att $(k_j - 1)T < b_j \leq k_j T$. Definiera ett schema (σ, t) genom att tilldela jobb j maskin $\sigma(j) = k_j$ med starttid $t_j = b_j - (k_j - 1)T - p_j$ (se Figur 1b).

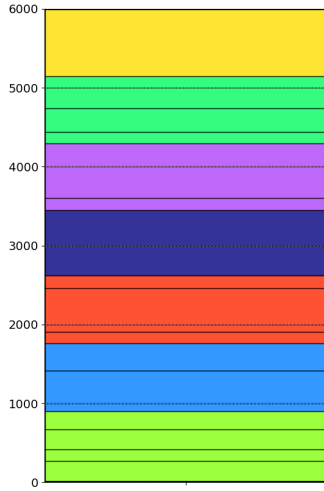
Eftersom brytpunkter har satts vid $T, 2T, \dots, mT$, sammanfaller sluttiden för det sista jobbet på varje maskin exakt med T . Detta resulterar i att (σ, t) är helt kompakt. Vidare garanteras att jobb från samma klass aldrig bearbetas parallellt eftersom jobb tillhörande samma klass är placerade efter varandra i schemat (σ, t) och klassernas bearbetningstider är begränsade till intervallet $[1, T]$. Restriktionerna $(\diamond)$ och $(\star)$ är därmed uppfyllda, och schemat (σ, t) har ett makespan på T .

Slutligen medför testgeneratorns konstruktion att $\frac{1}{m} \sum_{j=1}^n p_j = T$. Detta innebär att schemats makespan sammanfaller med en teoretisk undre gräns för problemet, eftersom det optimala makespan-värdet inte kan understiga summan av jobbets bearbetningstider delat på antalet maskiner. Av detta följer att schemat (σ, t) är optimalt. $\square$

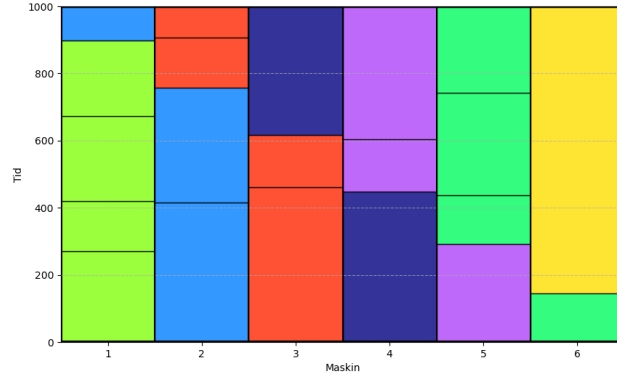
5.1.2 Schema för generering av testfiler

För att kunna utvärdera programmet och dess optimerade versioner under olika parameterförhållanden och instansstorlekar, genereras testfiler utifrån ett antal testserier. Varje testserie har en gemensam startinstans, varefter parametervärdena ökar stegvis enligt Tabell 1. Med undantag för serie g och h utgår samtliga testserier från startinstansen $m_1 = 3, K_1 = 5, n_1 = 10$. Undantagen g och h har startinstanserna $m_1 = 1, K_1 = 10, n_1 = 30$ respektive $m_1 = 15, K_1 = 30, n_1 = 90$.

En testfil betecknas i_j , där i anger testserien och j anger instansens ordningsnummer i serien. Exempelvis innebär testfil a_3 att det är det tredje steget i serie a , med $m = 3$ maskiner, $K = 5$



(a) Partitionering av intervallet $[0, W]$. Den totala arbetsmängden $W = mT = 6000$ är uppdelad i 21 jobb fördelade över 7 klasser.



(b) Det optimala schemat (σ, t) som erhålls genom att placera intervallet $[1000(i-1), 1000i] \subseteq [0, W]$ på maskin i , där $i = 1, \dots, 6$.

Figur 1: Visualisering av testgeneratoren då $n = 21, K = 7, m = 6$ och $T = 1000$. Olika färger representerar olika klasser.

klasser och $n = 10 + 2 + 2 = 14$ jobb. På samma sätt innebär testfil c_2 att $m = 3 + 1 = 4$, $K = \lfloor \frac{5}{3} \cdot 4 \rfloor = 6$ och $n = 3 \cdot 6 = 18$. För varje serie genereras successivt större testfiler tills dess att körtiden för någon av programversionerna når begränsningen på 10 minuter.

I Avsnitt 5.2.1 används testserierna a – g för att utvärdera hur olika parameterförhållanden påverkar prestandan hos programmet och dess optimerade versioner. Därefter, i Avsnitt 5.2.2, används testserie h för att undersöka hur en exakt lösningsmetod står sig mot heuristiken simulated annealing för stora instanser.

Test	Parameterförändring	Syfte
a	$n = n + 2$	Liten ökning av antalet jobb
b	$K = K + 2, n = 3K$	Stort antal jobb relativt maskiner och klasser
c	$m = m + 1, K = \lfloor \frac{5}{3}m \rfloor, n = 3K$	Proportionell ökning av samtliga parametrar
d	$m = m + 1, K = K + 1, n = n + 1$	Lika stor ökning av alla parametrar
e	$m = m + 1, K = 5K, n = 7n$	Snabb skillnad
f	$m = 2m, K = m + 1, n = 3K$	Snabb ökning av maskiner vid få klasser
g	$m = m_1 + k, k = 1, 2, 3, \dots$	Antalet maskiner närmar sig antalet klasser
h	$m = m + 5, K = K + 10, n = 3K$	Generering av stora instanser

Tabell 1: Beskrivning av parameterförändringar samt syfte för testserierna a – h

5.1.3 Simulated annealing

För att kunna utvärdera när en exakt lösningsmetod är att föredra framför en approximativ, används som tidigare nämnt heuristiken *simulated annealing* som jämförelseobjekt. Det är en heuristisk optimeringsalgoritm som introducerades 1983 [13]. Algoritmen utforskar problemets sökrum med målet att finna en globalt optimal lösning. Till skillnad från enklare heuristiker undviker simulated annealing att fastna i lokala optima genom att, med viss sannolikhet, även acceptera steg som tillfälligt försämrar lösningen. Denna sannolikhet regleras av en avkylningsfaktor, $\alpha \in (0, 1)$,

som gradvis sänker acceptansgraden för sämre lösningar över tid. Denna mekanism gör det möjligt för algoritmen att inledningsvis utforska sökrymden brett, för att sedan konvergera mot en lösning som förhoppningsvis är nära det globala optimumet. Eftersom algoritmen är heuristisk innebär det att den, till skillnad från de exakta lösningsmetoderna, inte kan garantera att den lösning som hittas faktiskt är optimal.

Den C++-implementation av simulated annealing som används i prestandatesterna är hämtad från [12]. I denna implementation är avkylningsfaktorn $\alpha = 0.99999$ och antalet iterationer satt till 500 000. Ett högt värde på α i kombination med många iterationer innebär att algoritmen utforskar sökrymden noggrant, vilket minskar risken för att fastna i lokala optima. Det leder generellt till en bättre lösningskvalitet, men på bekostnad av längre körtid.

5.2 Resultat

Utifrån den metodologi och testdata som definierats ovan, presenteras och visualiseras här ett urval av resultaten från prestandatesterna. Samtliga testresultat finns sammanställda i tabellform i Appendix A.2.

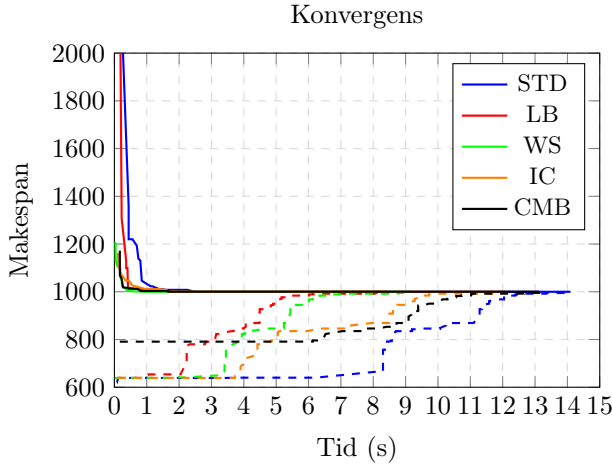
För att underlätta läsningen i grafer och tabeller benämns lösningsmetoderna enligt följande: standardmodellen (STD, presenterad i Avsnitt 3), modellen med lägre begränsning (LB, Avsnitt 4.1), modellen med warm start (WS, Avsnitt 4.2), modellen med indicator constraints (IC, Avsnitt 4.3), modellen med combined (CMB, Avsnitt 4.4) samt simulated annealing (SA, Avsnitt 5.1.3).

Under Gurobis lösningsprocess samlas kontinuerligt data in som möjliggör analys av hur lösningen utvecklas över tid. De två centrala värdena som spåras är *Incumbent* och *Best Bound*. Incumbent är den bästa möjliga lösning som hittills har hittats, det vill säga det lägsta makespan för vilket ett fullständigt giltigt schema existerar. Detta värde kan endast minska över tid i takt med att bättre lösningar hittas. Best Bound är en undre gräns för optimala makespan, det vill säga ett värde som den optimala lösningen garanterat inte kan understiga. Detta värde kan endast öka över tid allteftersom lösaren utesluter delar av sökrymden. Skillnaden mellan Incumbent och Best Bound anger hur långt från optimum den nuvarande lösningen som mest kan befinna sig. När de två värdena konvergerar och skillnaden når noll är den optimala lösningen bevisad. Om tidsgränsen nås innan dess returnerar Gurobi den bästa hittade lösningen tillsammans med den återstående skillnaden, vilket anger den garanterade övre gränsen för avståndet till den optimala lösningen.

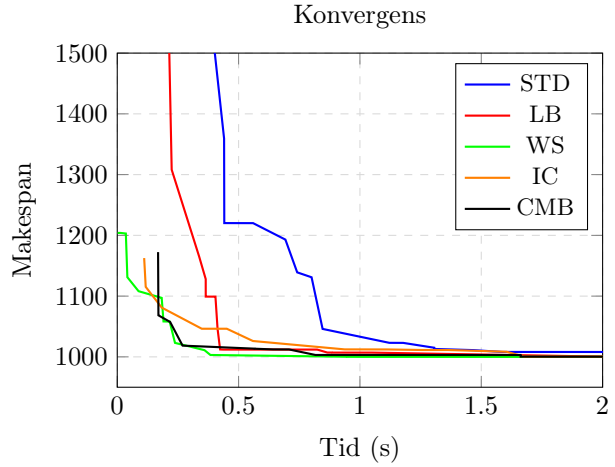
I graferna nedan illustreras lösningsprocessen för varje testinstans, där Incumbent visas som hel-dragen linje och Best Bound som streckad linje. Graferna visar hur den bästa kända lösningen förbättras och hur den undre gränsen stiger över tid tills de möts eller tidsgränsen nås [8].

5.2.1 Effekt av optimeringar

I detta avsnitt presenteras grafer över de mest intressanta resultaten för testerna *a-g*, samt ett stapeldiagram som sammanfattar dessa baserat på vilken metod som snabbast når makespan ≤ 1001 , snabbast når en undre begränsning på 1000 samt ger störst undre gräns om inte 1000 uppnås inom tidsbegränsningen. I Figur 2 till 8 tydliggör grafen till vänster utvecklingen av Best Bound medan grafen till höger är inzoomad för att illustrera utvecklingen av Incumbent bättre.

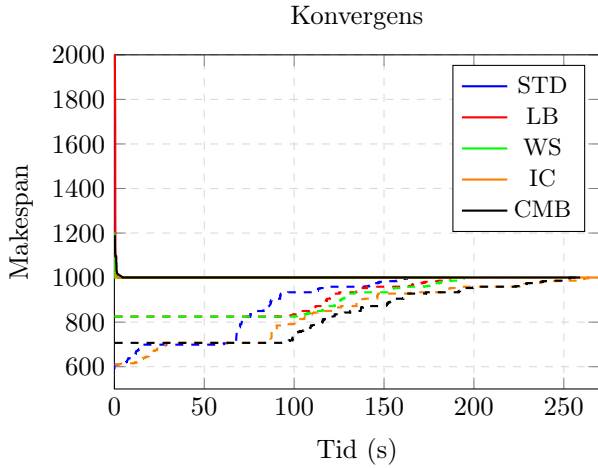


(a) Hela lösningsprocessen för test a_1 .

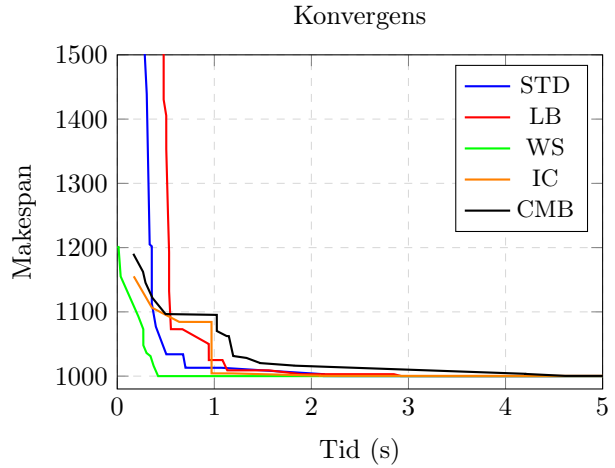


(b) Inzoomad bild av början på lösningsprocessen för test a_1 .

Figur 2: Test a_1 med $m = 3, K = 5, n = 10$, testar liten ökning av antal jobb. I test a_1 är WS snabbast att nå ett makespan på 1000, medan STD är långsammast. IC och CMB uppvisar liknande prestanda och placerar sig däremellan. Samma mönster gäller för utvecklingen av den undre gränsen, med undantag för att LB där är snabbast.

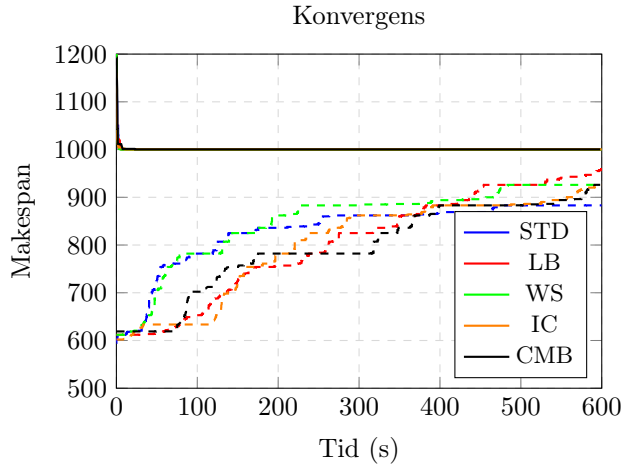


(a) Hela lösningsprocessen för test a_2 .

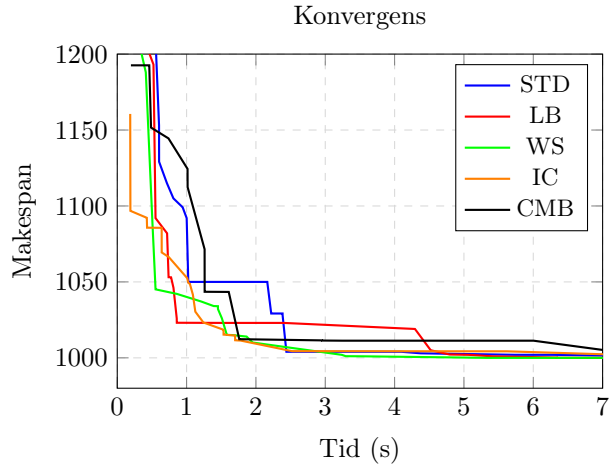


(b) Inzoomad bild av början på lösningsprocessen för test a_2 .

Figur 3: Test a_2 med $m = 3, K = 5, n = 12$, testar liten ökning av antal jobb. I test a_2 når samtliga metoder ett makespan på 1000, där WS är snabbast och CMB långsammast. Vad gäller den undre gränsen är STD och LB snabbast, medan IC och CMB kräver betydligt mer tid för att nå samma värde.

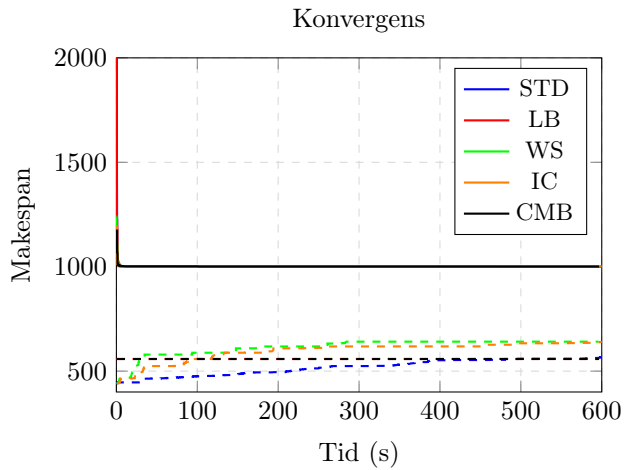


(a) Lösningsprocessen för test a_3 till och med tidsgränsen 600 sekunder.

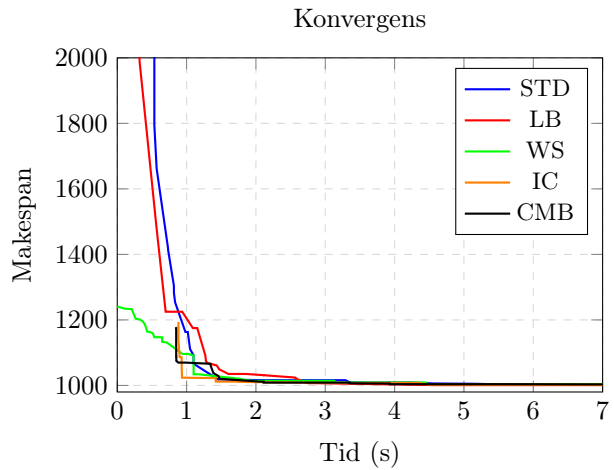


(b) Inzoomad bild av början på lösningsprocessen för test a_3 .

Figur 4: Test a_3 med $m = 3, K = 5, n = 14$, testar liten ökning av antal jobb. I test a_3 når samtliga metoder ett makespan på 1000, där WS är snabbast och CMB långsammast. Ingen av metoderna lyckas bevisa optimalitet inom tidsgränsen. WS, IC och CMB uppnår den högsta undre gränsen, där CMB gör det snabbast.

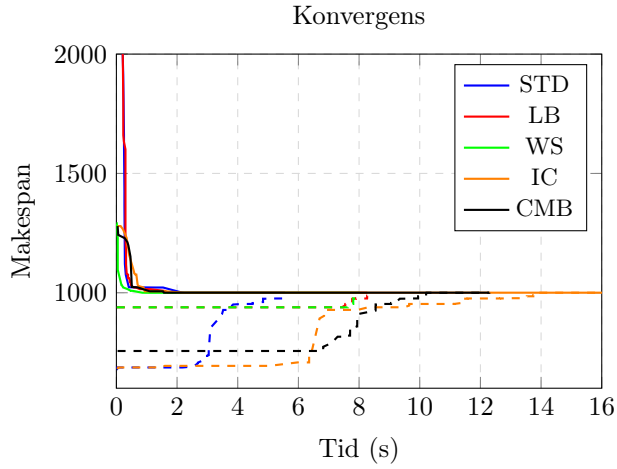


(a) Lösningsprocessen för test b_2 till och med tidsgränsen på 600 sekunder.

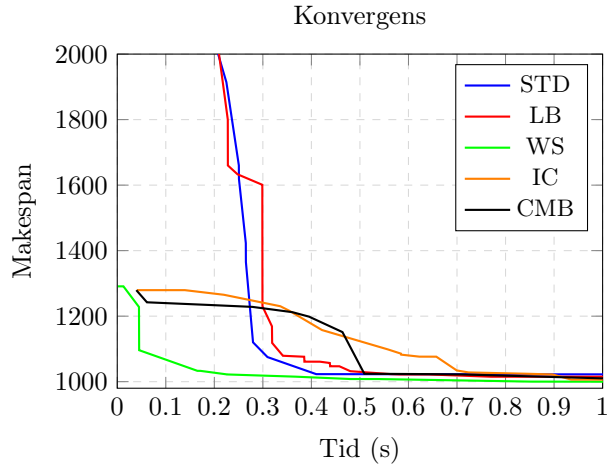


(b) Inzoomad bild av början på lösningsprocessen för test b_2 .

Figur 5: Test b_2 med $m = 3, K = 2, n = 21$, testar stort antal jobb relativt maskiner och klasser. I test b_2 hittade ingen metod en optimal lösning inom tidsgränsen, men LB når ett makespan på ≤ 1001 snabbast medan STD är långsammast. WS ger den högsta undre begränsningen medan LB och CMB ger den lägsta.

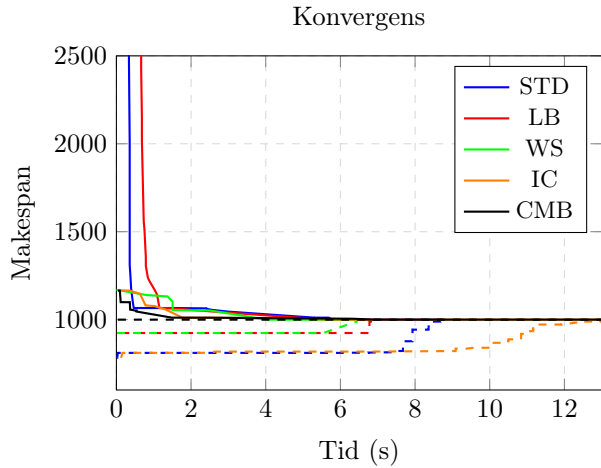


(a) Hela lösningsprocessen för test d_2 .

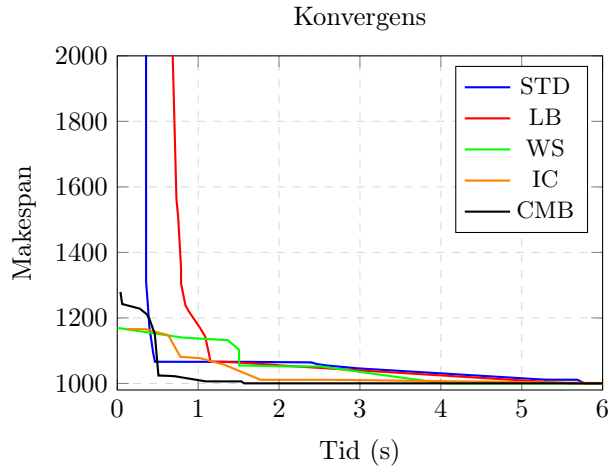


(b) Inzoomad bild på början av lösningsprocessen för test d_2 .

Figur 6: Test d_2 med $m = 4, K = 6, n = 11$ testar lika stor ökning av alla parametrar. I test d_2 når samtliga metoder ett makespan på ≤ 1001 och lyckas även bevisa optimalitet inom tidsgränsen. WS är snabbast att nå makespan medan STD, IC och CMB är långsammare. Vad gäller den undre gränsen är STD snabbast medan CMB kräver betydligt mer tid än övriga metoder.

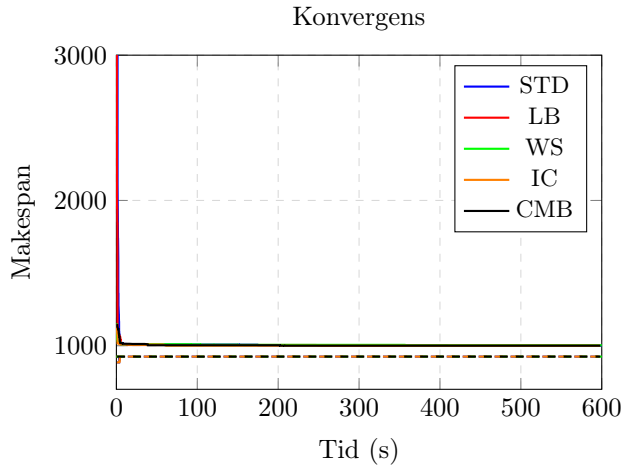


(a) Hela lösningsprocessen för test d_4 .

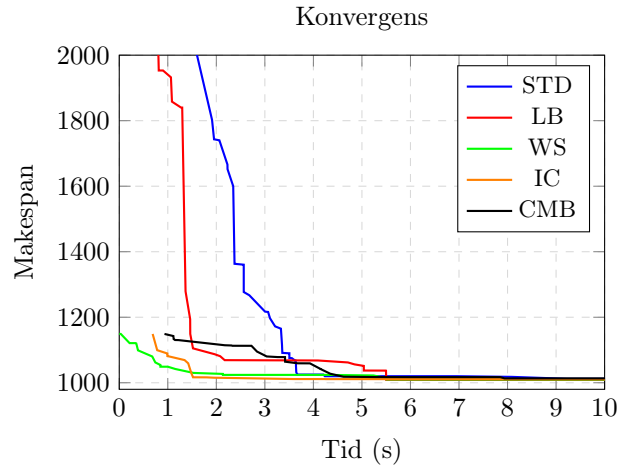


(b) Inzoomad bild på början av lösningsprocessen för test d_4 .

Figur 7: Test d_4 med $m = 6, K = 8, n = 13$ testar lika stor ökning av alla parametrar. I test d_4 når samtliga metoder ett makespan på 1000 och bevisar optimalitet inom tidsgränsen. WS är snabbast att nå makespan medan CMB är långsammast. Vad gäller den undre gränsen är CMB ensam om att nå värdet 1000 direkt vid start, medan övriga metoder kräver längre tid.

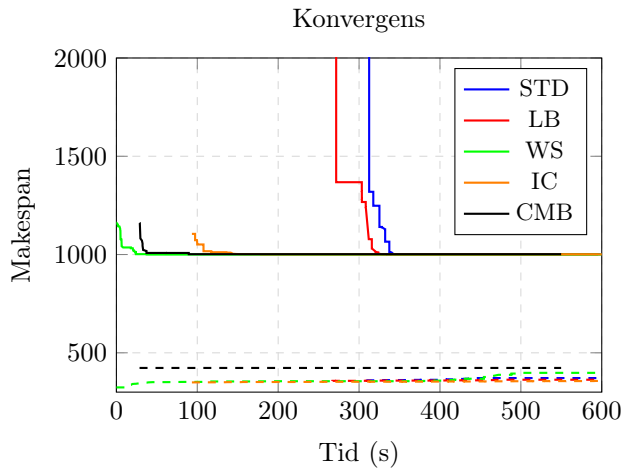


(a) Hela lösningsprocessen för test f_2 till och med tidsbegränsningen på 600 sekunder.

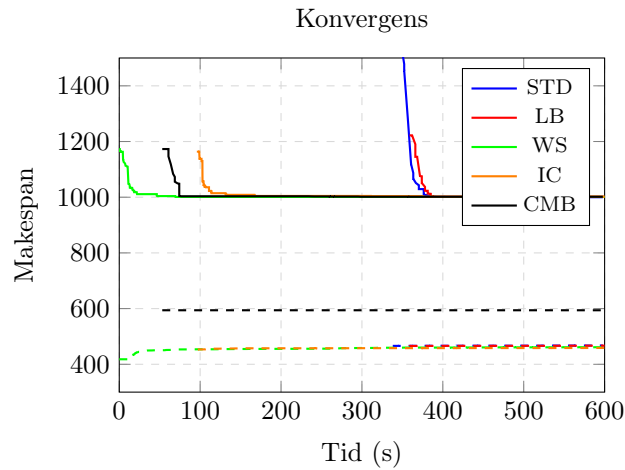


(b) Inzoomad bild på början av lösningsprocessen för test f_2 .

Figur 8: Test f_2 med $m = 6, K = 7, n = 21$, testar snabb ökning av maskiner vid få klasser. I test f_2 når STD och LB ett makespan på 1000 men kräver lång tid, medan WS inte lyckas nå under 1001 inom tidsgränsen. IC når makespan ≤ 1001 snabbare än övriga metoder. Ingen av metoderna lyckas bevisa optimalitet, och LB samt WS når sin bästa undre gräns direkt vid start.

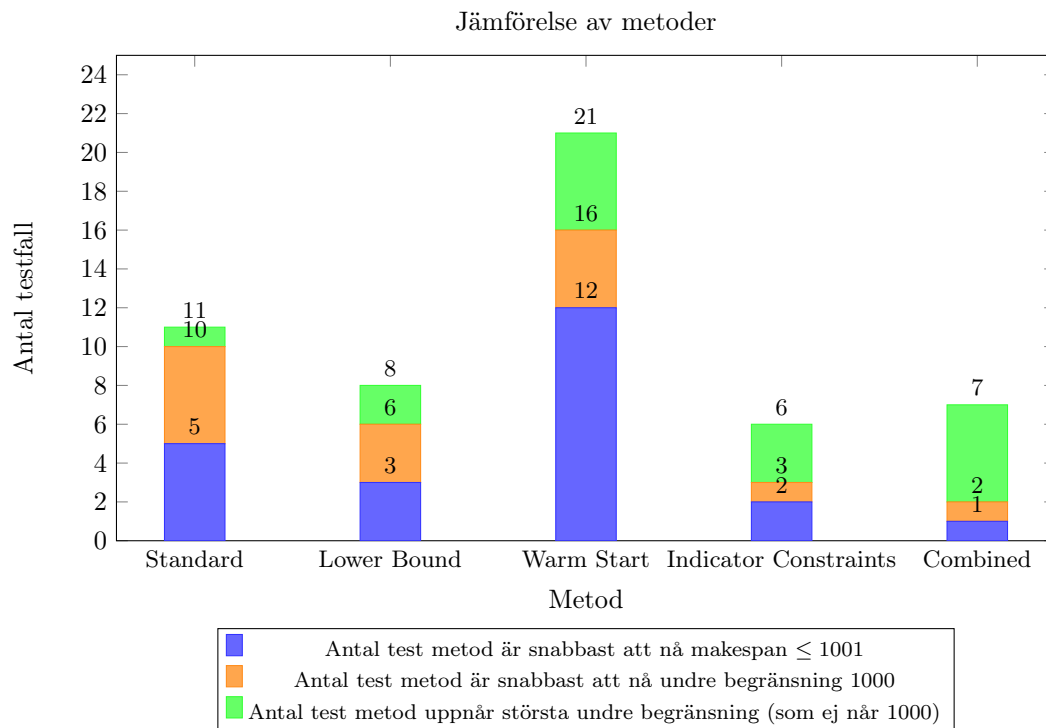


(a) Hela lösningsprocessen för test g_3 till och med tidsgränsen på 600 sekunder.



(b) Hela lösningsprocessen för test g_4 till och med tidsgränsen på 600 sekunder.

Figur 9: Test g_3 med $m = 3, K = 10, n = 30$ respektive g_4 med $m = 4, K = 10, n = 30$ testar då antalet maskiner närmar sig antalet klasser. I test g_3 hittar WS makespan 1000 betydligt snabbare än övriga, medan STD och LB kräver avsevärt längre tid. CMB har högsta undre gränsen, men ingen metod bevisar optimalitet inom tidsgränsen. I test g_4 är mönstret liknande: WS når makespan ≤ 1001 relativt snabbt, STD kräver avsevärt längre tid, och LB, IC samt CMB når aldrig under 1001. CMB har återigen högsta undre gränsen, utan att optimalitet bevisas.



Figur 10: Sammanställning av resultat för tester *a-g*, där WS dominerar genom att vara snabbast att nå ett makespan ≤ 1001 i flest testfall, medan IC och CMB presterar sämst. Exakta värden för samtliga tester finns i appendix A.2.

5.2.2 Prestanda för warm start och simulated annealing vid stora instanser

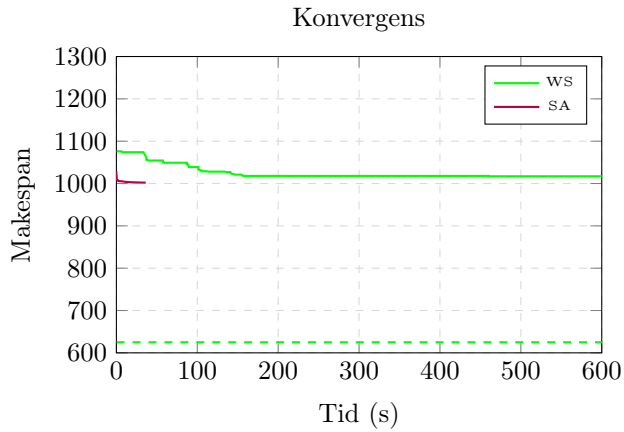
Som framgår av Figur 10 är WS den mest effektiva optimeringen. I detta avsnitt presenteras därför resultaten för WS tillsammans med heuristiken SA för test *h* (stora instanser). Figur 11 illustrerar de två lösningsmetodernas prestanda avseende lösningskvalitet och körtid. Kurvan för SA illustrerar en brant och snabb förbättring i början av lösningsprocessen, medan WS-kurvan visar ett mycket planare och långsammare förlopp.

5.3 Diskussion

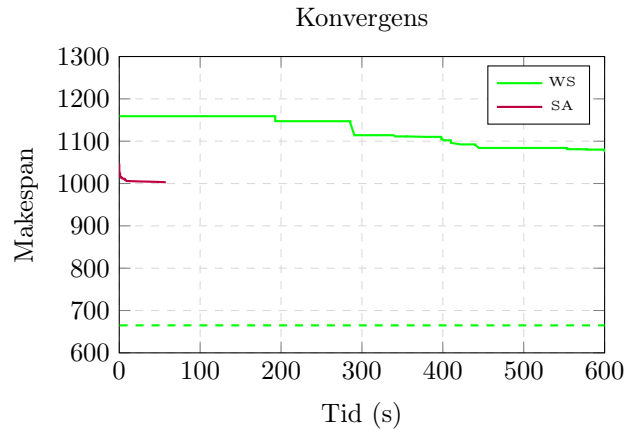
I detta avsnitt diskuteras och analyseras resultaten från Avsnitt 5.2, baserat på både konvergensplot och de sammanställda tabellerna i Appendix A.2. Inledningsvis utvärderas vilken av de implementerade optimeringsmetoderna som presterar bäst sett till lösningskvalitet och körtid. Därefter undersöks hur variation i bearbetningstider påverkar körtiden, följt av en diskussion om avvägningen mellan exakta och approximativa lösningsmetoder för storskaliga problem.

5.3.1 Den bästa optimeringsmetoden

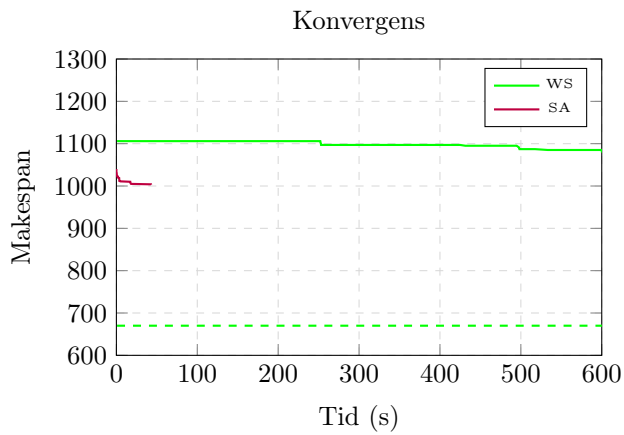
Trots att kombinerade optimeringsmetoden förväntades prestera bäst indikerar resultaten att WS ger den bästa balansen mellan lösningskvalitet och körtid av de implementerade optimeringarna. Som framgår i Figur 10 var denna metod snabbast med att nå makespan 1000 och hitta störst undre begränsning för de flesta testerna, vilket i Gurobi säkerställer att lösningen är optimal. En faktor som verkar ha en betydande effekt på lösningsprocessens effektivitet är det initiala startvärdet på makespan som WS förses med. Gurobi ges redan från start en genomförbar lösning att utgå ifrån, vilket påskyndar sökningen. Detta blir särskilt tydligt i Figur 9 där WS påbörjar sin konvergens betydligt tidigare än övriga metoder i både test g_3 och g_4 . Det är även extra tydligt att WS har god effekt för större instanser, exempelvis test e_2 , där övriga optimeringsmetoder inte lyckas hitta någon genomförbar lösning alls inom tidsgränsen.



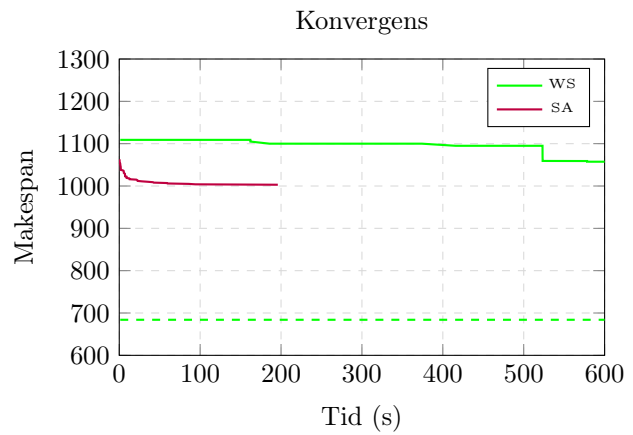
(a) I test h_1 ($n = 90, K = 30, m = 15$) nås bästa makespan på 1002 för SA (36 s) och 1017 för WS (460 s).



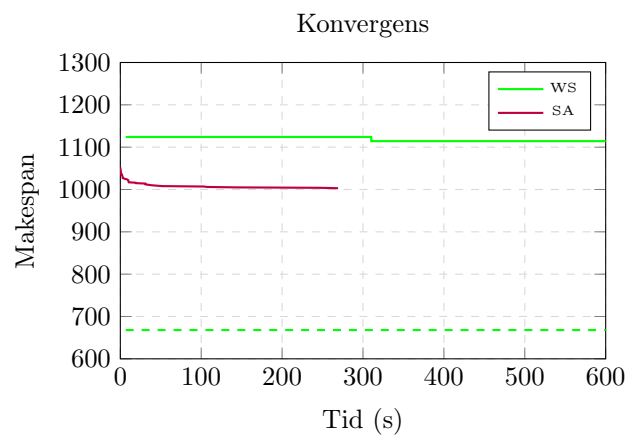
(b) I test h_2 ($n = 120, K = 40, m = 20$) nås bästa makespan på 1003 för SA (58 s) och 1074 för WS (600 s).



(c) I test h_3 ($n = 150, K = 50, m = 25$) nås bästa makespan på 1003 för SA (43 s) och 1079 för WS (600 s).



(d) I test h_4 ($n = 180, K = 60, m = 30$) nås bästa makespan på 1003 för SA (196 s) och 1057 för WS (583 s).



(e) I test h_5 ($n = 210, K = 70, m = 35$) nås bästa makespan på 1003 för SA (269 s) och 1114 för WS (310 s).

Figur 11: Jämförelse mellan warm start (WS) och simulated annealing (SA). Graferna visar den bästa funna lösningen över tid. Sökningen avbryts när det maximala antalet iterationer uppnås (SA), respektive vid tidsgränsen 600 s (WS).

En möjlig förklaring till att WS presterar bättre än CMB är att WS redan minskar sökrymden markant genom en stark övre gräns och en bra initial lösning. Det kan vara så att adderingen av indicator constraints tar bort effekten av ett tajtare big-M och därmed ger en ökad beräkningskostnad när Gurobi internt måste hantera de logiska villkoren.

En viktig aspekt att beakta är att WS och CMB kräver att en approximationsalgoritm körs innan ILP-modellen löses, vilket innebär en extra beräkningskostnad. Den tiden det tar att lösa approximationsalgoritmen har dessutom inte räknats med i resultaten för WS- eller CMB-testerna. Det gör att WS inte nödvändigtvis är det mest självklara valet, till exempel för mindre instansstorlekar där standardmetoden eller andra optimeringsmetoder också hittar en optimal lösning snabbt.

5.3.2 Variation i bearbetningstid och dess påverkan på körtiden

Testserierna $a-f$ utgår alla från samma startinstans $m_1 = 3$, $K_1 = 5$, $n_1 = 10$. Resultaten från testerna a_1-f_1 tyder på att körtiden inte enbart beror på instansstorlek, eftersom dessa tester uppvisar variation i lösningstid trots identiska grundparametrar. Det som skiljer dessa tester åt är variationen i bearbetningstider för jobben, vilket indikerar att även denna faktor påverkar körtiden. Baserat på lösningstiderna framgår till exempel att test c_1 löstes snabbast (0.9901 sekunder), medan f_1 löstes långsammast (21.2234 sekunder). Antalet utforskade noder var 1370 respektive 227981. Test c_1 var det testet med störst variation i bearbetningstider från det minsta jobbet på 42 tidsenheter till det största på 624 tidsenheter. Det är tydligt från testerna att instanser där bearbetningstiden skiljer sig åt kraftigt tenderar att lösas snabbare.

En möjlig förklaring till detta är att Gurobi bygger på en branch-and-bound-algoritm [11]. Vid stor spridning i bearbetningstider kan den linjära relaxation av modellen bli starkare, vilket ger bättre gränser (bounds). Detta leder i sin tur till att färre grenar (noder) behöver undersökas, vilket resulterar i kortare körtid. Med avseende på detta skulle en möjlig förbättring av testmetoden vara att testa alla testserier med alla olika startinstanser a_1-f_1 för att få ett mer exakt resultat.

5.3.3 Avvägning mellan exakt och approximativ lösningsmetod

Av Figur 11 framgår det tydligt att SA presterar avsevärt bättre än WS, avseende både lösningskvalitet och körtid. För test h_5 lyckas WS knappt förbättra sin initiala lösning under den tillåtna körtiden på 10 minuter (se Figur 11e). Utifrån resultaten är det troligt att WS hade krävt en orimligt lång beräkningstid för att hitta och bevisa en optimal lösning. Trots att SA saknar optimalitetsgaranti konvergerar den snabbt mot en betydligt bättre lösning, och därför är SA ett mer lämpligt val för storskaliga problem.

En trolig förklaring till simulated annealings överlägsna prestanda är dess relativt enkla struktur, vilket beror på att algoritmen saknar krav på att bevisa optimalitet. Branch-and-bound-algoritmen som Gurobi använder sig av måste däremot både hitta giltiga lösningar och beräkna undre gränser till problemet, vilket är en mycket tidskrävande process. I praktiska tillämpningar är beräkningstiden ofta begränsad, vilket gör att den algoritm som kan hitta bäst lösning på kortast tid oftast är att föredra. Detta gör att den exakta metodens optimalitetsgaranti har mindre relevans för stora problem, eftersom en heuristisk lösningsmetod kan hitta betydligt bättre lösningar inom samma begränsade tidsram.

6 Vidare forskning

Ytterligare optimeringar av programmet som inte har testats systematiskt, men som kan vara av intresse, diskuteras i detta avsnitt.

6.1 Symmetrier i ILP-formuleringen

Resultaten i Avsnitt 5.2 visar att Gurobi ofta snabbt hittar en lösning med optimalt makespan, men kräver lång tid att öka undre gränsen och därmed bevisa optimalitet. En tänkbar förklaring till detta är förekomsten av symmetrier i ILP-formuleringen från Avsnitt 3.2. Ett linjärt heltalsproblem (ILP) sägs vara symmetriskt om dess variabler kan permuteras utan att problemets struktur

förändras. Mer formellt definieras problemets symmetrigrupp som mängden av alla permutationer av variablerna som avbildar varje tillåten lösning på en annan tillåten lösning med exakt samma målfunktionsvärde [14].

Till exempel, antag att vi har två maskiner ($m = 2$), tre klasser ($K = 3$) och sex jobb ($n = 6$) med identiska bearbetningstider $p_1 = p_2 = \dots = p_6 = 10$. Om jobb 1 och 2 tillhör klass 1, jobb 3 och 4 tillhör klass 2 och jobb 5 och 6 tillhör klass 3 så ingår exempelvis permutationen $\pi = (1\ 2)(3\ 4)(5\ 6)$ i problemets symmetrigrupp. Detta beror på att jobb inom samma klass är identiska och kan byta plats med varandra utan att schemat blir ogiltigt eller makespan förändras.

Konsekvensen av detta är att Gurobi behöver utforska många strukturellt identiska lösningar, vilket gör att den undre gränsen ökar mycket långsamt för större instanser. Strategier för att hantera och bryta sådana symmetrier beskrivs i [14]. Genom att implementera någon av dessa metoder i programmet i Avsnitt 3, hade tiden för att bevisa optimalitet potentiellt kunnat förkortas när en optimal lösning väl hittats.

6.2 Vidare optimering av Warm Start

En annan möjlig förbättring är att komplettera WS-algoritmen med så kallade *variable hints*. Till skillnad från MIP-start, som används för att generera en initial möjlig lösning, påverkar hints hela optimeringsprocessen genom att vägleda lösarens heuristiker och förgreningsbeslut i sökträdet [9]. Konkret kan hints sättas för de binära variablerna $x_{j,j'}$, $y_{j,j'}$ och $z_{j,j'}$ genom att härledas direkt från approximationslösningens starttider t_j enligt de logiska villkoren $x_{j,j'} = 1$ om $t_{j'} \leq t_j$, $y_{j,j'} = 1$ om $t_j < t_{j'} + p_{j'}$ och $z_{j,j'} = 1$ om $t_j \in [t_{j'}, t_{j'} + p_{j'})$.

Högkvalitativa hints bör ge högkvalitativa MIP-lösningar snabbare, medan lågkvalitativa hints kan leda till onödigt arbete för lösaren, men inte till dramatisk prestandaförsämring [9]. En implementation av hints är framtiden och finns tillgänglig på GitHub² men har inte utvärderats systematiskt.

Ytterligare en möjlig förbättring är att förse Gurobi med flera MIP-starts från olika approximationsalgoritmer. Gurobi stöder detta via modellattributet `NumStart` [9]. Eftersom de olika approximationsalgoritmerna sannolikt producerar strukturellt olika scheman skulle detta sprida startpunkterna över sökrymden och öka sannolikheten att en av dem ligger nära den optimala lösningen. Huruvida dessa förbättringar, enskilt eller i kombination, ger en mätbar förbättring av lösningstiden återstår att undersöka.

Referenser

- [1] Jacek Blazewicz, Jan K. Lenstra och Alexander H. G. Rinnooy Kan. “Scheduling subject to resource constraints: classification and complexity”. I: *Discrete Applied Mathematics* 5.1 (1983), s. 11–24. DOI: [https://doi.org/10.1016/0166-218X\(83\)90012-4](https://doi.org/10.1016/0166-218X(83)90012-4).
- [2] Copernicus. *Security*. European Union’s Earth Observation Programme. Hämtad: 2026-03-23. URL: <https://www.copernicus.eu/en/copernicus-services/security>.
- [3] Max A. Deppert m. fl. “Scheduling with many shared resources”. I: *Proceedings of the IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2023, s. 413–423.
- [4] Gyorgy Dosa, Hans Kellerer och Zsolt Tuza. “Restricted assignment scheduling with resource constraints”. I: *Theoretical Computer Science* 760 (2019), s. 72–87. DOI: <https://doi.org/10.1016/j.tcs.2018.08.016>.
- [5] ESA. *ESA Space Environment Report 2025*. European Space Agency. Hämtad: 2026-03-23. April 2025. URL: https://www.esa.int/Space_Safety/Space_Debris/ESA_Space_Environment_Report_2025.
- [6] EUR-Lex. *Copernicus*. Europeiska unionens publikationsbyrå. Hämtad: 2026-03-23. URL: <https://eur-lex.europa.eu/SV/legal-content/glossary/copernicus.html>.

²<https://github.com/almajolin/Bachelor-Thesis-MSRS>

- [7] Gurobi Optimization, LLC. *Constraints*. Hämtad: 2026-04-19. URL: <https://docs.gurobi.com/projects/optimizer/en/current/concepts/modeling/constraints.html#general-constraints>.
- [8] Gurobi Optimization, LLC. *Gurobi Optimizer Reference Manual – MIP Logging*. Accessed: 2026-04-30. 2026. URL: <https://docs.gurobi.com/projects/optimizer/en/current/concepts/logging/mip.html>.
- [9] Gurobi Optimization, LLC. *Start – Gurobi Optimizer Reference Manual*. Hämtad: 2026-04-14. URL: <https://docs.gurobi.com/projects/optimizer/en/current/reference/attributes/variable.html#start>.
- [10] Emmanuel Hebrard m. fl. “Approximation of the parallel machine scheduling problem with additional unit resources”. I: *Discrete Applied Mathematics* 215 (2016), s. 126–135. DOI: <https://doi.org/10.1016/j.dam.2016.07.003>.
- [11] Lundgren J., Rönnqvist M. och Värbrand P. “Trädsökning”. I: *Optimeringslära*. 3. utg. Studentlitteratur, 2008. Kap. 15, s. 393–419. ISBN: 9789144053141.
- [12] Hampus Kihl, Olle Björkenbacke och Olle Lapidus. *Many Shared Resources Scheduling Problem: Improvements and Implementations of Approximation Algorithms and Heuristics*. Bachelor’s thesis. 2025.
- [13] S. Kirkpatrick, C. D. Gelatt Jr. och M. P. Vecchi. “Optimization by Simulated Annealing”. I: *Science* 220.4598 (1983), s. 671–680.
- [14] François Margot. “Symmetry in Integer Linear Programming”. I: *50 Years of Integer Programming 1958-2008*. Utg. av M. Jünger m. fl. Springer-Verlag Berlin Heidelberg, 2010, s. 647–686. DOI: [10.1007/978-3-540-68279-0_17](https://doi.org/10.1007/978-3-540-68279-0_17).
- [15] Rymdstyrelsen. *Copernicus 25 år*. Hämtad: 2026-03-23. Juni 2023. URL: <https://www.rymdstyrelsen.se/upptack-rymden/bloggen/2023/06/copernicus-25-ar/>.
- [16] Vitaly A. Strusevich. “Approximation algorithms for makespan minimization on identical parallel machines under resource constraints”. I: *Journal of the Operational Research Society* 72.9 (2021), s. 2135–2146. DOI: <https://doi.org/10.1080/01605682.2020.1772019>.

AI-användning

Generativ AI har använts i begränsad omfattning och med stor försiktighet under arbetets gång. Vid implementation och kodutveckling har AI nyttjats som ett stöd, där genererad kod noggrant granskats och verifierats för att säkerställa korrekthet. AI har även använts som hjälpmedel i skrivprocessen, bland annat för formuleringar, synonymer och språklig korrekturläsning, samt för formatering av tabeller och grafer i LaTeX. Utöver detta har AI använts för att få förklaringar av begrepp och som ett stöd vid sökning efter lämpliga källor, där samtliga källor därefter kontrollerats och bedömts självständigt. Användningen har genomgående skett i situationer där det bedömts kunna effektivisera arbetet utan att kompromissa med kvaliteten, och allt AI-genererat innehåll har granskats kritiskt innan det inkluderats i rapporten. Målet har varit att använda AI som ett hjälpmedel som stödjer arbetsprocessen, snarare än en ersättning för eget arbete, egna bedömningar och analys.

A Appendix

A.1 Pseudokod för testgeneratoren

Algoritm 2: Testgenerator

Indata: Antal jobb n , klasser K , maskiner m och makespan T **Utdata:** Körtid p_j och klass c_j för varje jobb j $W \leftarrow m \cdot T$ // Steg 1: Partitionera $[0, W]$ i K klassintervall $\mathbf{t} \leftarrow \mathbf{1}_K$ $W_{res} \leftarrow W - \text{sum}(\mathbf{t})$ $\mu \leftarrow \frac{W}{K}$ **while** $W_{res} > 0$ **do** $\pi \leftarrow \text{permute}(1, \dots, K)$ **for** $i \in \pi$ **do** **if** $W_{res} \leq 0$ **then** **break** $\lambda \leftarrow \mu - t_i$ **if** $t_i < T$ **and** $\lambda > 0$ **then** $\delta_{max} \leftarrow \min(W_{res}, T - t_i)$ **if** $\delta_{max} < \lambda$ **then** $\delta \leftarrow \delta_{max}$ **else** $\delta \sim \text{Poisson}(\lambda)$ begränsad till $[1, \delta_{max}]$ $t_i \leftarrow t_i + \delta$ $W_{res} \leftarrow W_{res} - \delta$ $\mathcal{C} \leftarrow \text{cumsum}(\mathbf{t})$

// Steg 2: Inkludera maskinbrytpunkter för att garantera kompakt schema

 $\mathcal{M} \leftarrow \{i \cdot T \mid i \in \{1, \dots, m\}\}$ $\mathcal{B} \leftarrow \mathcal{C} \cup \mathcal{M}$ // Steg 3: Justera antalet brytpunkter för att uppnå n jobb**if** $n < |\mathcal{B}|$ **then** $i \leftarrow 1$ **while** $n < |\mathcal{B}|$ **do** Ersätt i :te minsta elementet i $\mathcal{C}$ med $i \cdot T$ $i \leftarrow i + 1$ $\mathcal{B} \leftarrow \mathcal{M} \cup \mathcal{C}$ **else** **while** $n > |\mathcal{B}|$ **do** $b \leftarrow \text{slumpmässigt heltal från } \{1, \dots, W\}$ $\mathcal{B} \leftarrow \mathcal{B} \cup \{b\}$ // Steg 4: Generera p_j och c_j för varje jobb j Sortera $\mathcal{B}$ så att $b_1 < b_2 < \dots < b_n$ $b_0 \leftarrow 0$ $c \leftarrow 1$ **for** $j = 1, \dots, n$ **do** $p_j \leftarrow b_j - b_{j-1}$ $c_j \leftarrow c$ **if** $b_j \in \mathcal{C}$ **then** $c \leftarrow c + 1$ **return** $\{(p_j, c_j) \mid j = 1, \dots, n\}$

A.2 Testresultat

Tabell 2: Sammanställning av testresultat för serierna a – g . *Tid (lösning)* avser tiden i sekunder då en lösning med makespan ≤ 1001 hittades, och *Tid (undre gräns)* avser tiden i sekunder då den bästa undre gränsen uppnåddes.

Metod	Bästa lösning	Tid (lösning)	Bästa undre gräns	Tid (undre gräns)
Testserie a				
Test a_1				
STD	1000	2.489	1000	14.097
LB	1000	2.044	1000	8.464
WS	1000	0.945	1000	9.148
IC	1000	1.677	1000	12.072
CMB	1000	1.662	1000	13.131
Test a_2				
STD	1000	2.129	1000	162.752
LB	1000	2.924	1000	186.721
WS	1000	0.421	1000	194.360
IC	1000	2.087	1000	273.640
CMB	1000	4.613	1000	259.122
Test a_3				
STD	1000	7.771	883.100	580.727
LB	1000	5.420	962	600
WS	1000	3.573	926.100	600
IC	1000	10.698	926.100	594.856
CMB	1000	23.202	926.100	591.011
Testserie b				
Test b_1				
STD	1000	0.352	1000	3.490
LB	1000	0.252	1000	5.004
WS	1000	0.268	1000	2.740
IC	1000	0.914	1000	4.308
CMB	1000	0.293	1000	5.519
Test b_2				
STD	1000	72.189	568.274	600
LB	1000	4.572	558	0.3040
WS	1000	15.523	641	589.955
IC	1000	7.234	639.300	600
CMB	1000	11.781	558	0.848
Testserie c				
Test c_1				
STD	1000	0.204	1000	0.425
LB	1000	0.429	1000	0.429
WS	1000	0.427	1000	0.569
IC	1000.3	0.509	1000	0.865
CMB	1000.3	0.347	1000	0.886
Test c_2				
STD	1000	5.487	947	575.461
LB	1000	14.836	947	317.529
WS	1000	5.500	960	427.911
IC	1000.6	20.559	948.200	575.790

Fortsätter på nästa sida...

Tabell 2 – fortsättning från föregående sida

Metod	Bästa lösning	Tid (lösning)	Bästa undre gräns	Tid (undre gräns)
CMB	1000.6	187.493	947.1	490.081
Testserie d				
Test d_1				
STD	1000	1.184	1000	20.371
LB	1000	1.305	1000	20.914
WS	1000	0.318	1000	12.175
IC	1000	2.368	1000	42.202
CMB	1000	1.567	1000	259.122
Test d_2				
STD	1000	2.180	1000	5.558
LB	1000	1.707	1000	8.272
WS	1000	0.852	1000	8.020
IC	1000.4	2.183	1000	13.849
CMB	1000.4	2.869	1000	25.044
Test d_3				
STD	1000	2.676	1000	4.799
LB	1329	-	133	0.095
WS	1329	-	1329	0.039
IC	1000.2	1.995	1000	7.773
CMB	1000.2	3.0426	1000	13.339
Test d_4				
STD	1000	5.779	1000	8.717
LB	1000	5.601	1000	6.818
WS	1000	4.027	1000	6.431
IC	1000	6.331	1000	13.128
CMB	1000	6.857	1000	0
Test d_6				
STD	1000	1.876	1000	0
LB	1000	2.069	1000	0
WS	1161	-	1161	0.026
IC	1000.2	7.271	1000	0
CMB	1000.2	7.346	1000	0
Testserie e				
Test e_1				
STD	1000	0.216	1000	1.802
LB	1000	0.262	1000	2.467
WS	1000	0.250	1000	1.727
IC	1000	0.417	1000	2.170
CMB	1000	0.303	1000	2.473
Test e_2				
STD		Ingen lösning funnen.		
LB		Ingen lösning funnen.		
WS	1002	-	206.483	444.991
IC		Ingen lösning funnen.		
CMB		Ingen lösning funnen.		
Testserie f				
Test f_1				
STD	1000	1.147	1000	21.203

Fortsätter på nästa sida...

Tabell 2 – fortsättning från föregående sida

Metod	Bästa lösning	Tid (lösning)	Bästa undre gräns	Tid (undre gräns)
LB	1000	1.484	1000	10.021
WS	1000	0.645	1000	12.576
IC	1000.3	2.136	1000	15.786
CMB	1000.3	0.556	1000	15.210
Test f_2				
STD	1000	410.673	925	4.507
LB	1000	249.986	925	0
WS	1003	-	925	0
IC	1000.300	61.524	925.200	5.038
CMB	1000.300	207.447	925.200	158.330
Testserie g				
Test g_1				
STD	1000	0	601.200	179.135
LB	1000	0	622.400	600
WS	1000	0	629.400	600
IC		Ingen lösning funnen.		
CMB		Ingen lösning funnen.		
Test g_2				
STD	1000	94.877	318	555
LB	1000	154.440	325	218.389
WS	1000	1.956	325	404.531
IC	1001.400	-	342.900	443.500
CMB		Ingen lösning funnen.		
Test g_3				
STD	1002	-	371.500	440.030
LB	1001	340.168	363	469.672
WS	1000	37.021	398	499.696
IC	1001	178.522	357	590.126
CMB	1001.8	-	423	28.632
Test g_4				
STD	1000	401.824	467	534.805
LB	1002	-	467	544.988
WS	1001	69.822	461	441.162
IC	1001.600	-	459	573.222
CMB	1001.6	-	594	53.164
Test g_5				
STD		Ingen lösning funnen.		
LB		Ingen lösning funnen.		
WS	1000	63.582	538.600	573.750
IC		Ingen lösning funnen.		
CMB	1001.5	-	621.0	0
Test g_9				
STD	1008	-	937	14.911
LB	1010	-	937	0
WS	1001	304.567	937	0
IC	1005	-	937	149.146
CMB	1023.2	-	937.1999	394.865

Tabell 3: Sammanställning av testresultat för serie h . *Tid (lösning)* avser tiden i sekunder då den bästa lösningen hittades.

Metod	Bästa lösning	Tid (lösning)
Test h_1		
WS	1017.100	460.108
SA	1002	36.285
Test h_2		
WS	1074.100	600.013
SA	1003	57.547
Test h_3		
WS	1079	600.251
SA	1003	42.973
Test h_4		
WS	1057.400	582.974
SA	1003	196.019
Test h_5		
WS	1114.200	310.243
SA	1003	269.210